

Prédiction du risque de défaut de crédit

Data Tour Afrique 2025 - Rapport complet

Résumé

Ce rapport documente un pipeline complet de scoring de crédit (17,6M lignes, 2,9M clients) : exploration, feature engineering, une démonstration explicite de deux pièges de fuite de données fréquents sur ce type de jeu de données, un modèle LightGBM simple, et son interprétation. Résultat retenu : ROC AUC $\approx$ 0.685 en validation, un score réaliste plutôt qu'artificiellement gonflé.

1. Contexte et objectif

Ce projet s'inscrit dans le cadre du Data Tour 2025, une compétition panafricaine de data science, sur un problème de credit scoring : à partir de l'historique de produits de crédit d'un client, il s'agit de prédire la probabilité qu'un prêt donné fasse défaut. Ce travail reprend et améliore, après la compétition, une première version du projet, en intégrant des retours d'expérience et de bonnes pratiques de scoring identifiées a posteriori.

L'objectif n'est pas de maximiser le score à tout prix, mais de trouver un juste milieu entre simplicité et performance : un pipeline reproductible, sans fuite de données.

2. Données

Le jeu de données brut compte 17 659 833 lignes (chacune représentant un produit de crédit) et 62 colonnes encodées (principalement en int8), pour 2 892 602 clients uniques. Il est fourni au format parquet (~350 Mo), découpé en 17 row groups d'environ 1 million de lignes chacun.

- Aucune valeur manquante ni doublon - vérifié sur l'intégralité des 17,6M lignes (et pas seulement un échantillon), y compris les doublons entre row groups différents, via un hachage par ligne.
- Cible flag très déséquilibrée : 3,50% de défauts (96,50% de non-défauts).
- Constat central : flag est identique pour toutes les lignes d'un même client (0 client avec un flag différent entre ses prêts). La cible est donc en réalité définie au niveau du client, pas du prêt. Ce constat conditionne toute la suite de la méthodologie (sections 3.3 et 5).

2.1 Lexique des variables

Le Data Tour 2025 a fourni aux participants un dictionnaire décrivant les principales variables du jeu de données. Les colonnes clés sont résumées ci-dessous.

Variable	Description
id	Identifiant unique de la demande de prêt.
flag	Variable cible (1 = défaut, 0 = remboursement normal).
rn	Numéro de série du produit de crédit dans l'historique du client.
pre_since_opened	Nombre de jours écoulés depuis l'ouverture du prêt.
pre_since_confirmed	Nombre de jours écoulés depuis la confirmation du prêt.
pre_pterm	Durée prévue du prêt.
pre_fterm	Durée réelle du prêt.
pre_loans_credit_limit	Limite de crédit accordée.
pre_loans_next_pay_summ	Montant de la prochaine échéance de paiement.
pre_loans_outstanding	Montant restant impayé (encours).
pre_loans_total_overdue	Montant en souffrance actuel.

Variable	Description
pre_loans_max_overdue_sum	Montant en souffrance maximal observé dans l'historique.
pre_util	Taux d'utilisation du crédit.
enc_paym_0 ... enc_paym_24	Statut de paiement mensuel encodé sur 25 mois (0 = à jour, valeur croissante = retard plus sévère).
client_*	Agrégats (moyenne / max / écart-type) calculés dans ce projet sur l'historique du client.

3. Méthodologie

3.1 Échantillonnage stratifié

Le jeu complet étant trop volumineux pour un pipeline rapide à exécuter, un échantillon stratifié par client est construit : 50 000 clients en défaut et 100 000 clients sans défaut (jamais un échantillonnage par ligne, pour ne pas couper l'historique d'un client), soit 150 000 clients et 903 011 lignes extraites. Le ratio obtenu (33,3%) est volontairement moins extrême que le ratio naturel (3,5%), pour accélérer l'entraînement. Le même pipeline s'applique tel quel au jeu complet.

3.2 Feature engineering

Trois familles de variables sont construites, en excluant au préalable 3 colonnes à risque de fuite (pre_fterm, pre_till_fclose, fclose_flag), connues seulement à la clôture d'un prêt :

- Historique de paiement (enc_paym_0 à enc_paym_24) : moyenne, écart-type, pire statut, nombre de mois en retard, tendance récente vs ancienne, et plus longue séquence consécutive de retard (calculée de façon vectorisée).
- Ratios financiers : encours/plafond, impayés/plafond, prochaine échéance/encours, dépassement/plafond => plus parlants pour un modèle que des montants bruts.
- Agrégats client (groupby id) : moyenne, max et écart-type des principales variables sur l'historique du client, plus le nombre de produits détenus. Cette famille s'avère la plus prédictive (sections 4.3 et 4.4).

Au total, le jeu de features compte 88 colonnes (86 variables explicatives, id et flag).

3.3 Validation : le piège du split naïf

Un piège fréquent sur ce type de données consiste à valider avec un split au niveau des lignes (par exemple un train_test_split classique stratifié sur flag). Or un même client peut apparaître dans plusieurs lignes très similaires : s'il se retrouve à la fois en train et en validation, le modèle peut « reconnaître » ce client plutôt que d'apprendre un vrai signal de risque.

Rappel : flag est identique pour toutes les lignes d'un même client (section 2).

Ce piège a été vérifié concrètement plutôt que supposé : sur un split stratifié par flag au niveau ligne, 90 254 clients (60%) et 711 232 lignes (79%) se retrouvent partagés entre train et validation. Le modèle entraîné sur ce split obtient une AUC de 0.7247 contre ~0.685 pour le modèle finale (section 4.1), soit un écart de +0.04, très supérieur à n'importe quel gain de réglage d'hyperparamètres observé dans ce projet (section 3.5).

Le split retenu est donc groupé par client (chaque client va entièrement en train ou en validation, jamais les deux) et stratifié par flag (calculé séparément sur les clients en défaut et sans défaut, pour garantir un taux de défaut comparable des deux côtés plutôt que de compter sur le hasard) :

Ensemble	Lignes	Clients	Taux de défaut
Train	721 965	≈ 120 000	0.323
Validation	181 046	≈ 30 000	0.324

3.4 Modèle : LightGBM

Le modèle retenu est un LightGBM unique (pas d'ensembling), avec des hyperparamètres par défaut : aucun réglage manuel arbitraire n'est appliqué. Seuls deux ajustements sont ajoutés, tous deux justifiables pour ce problème : `scale_pos_weight` (2.092) pour compenser le déséquilibre des classes, et l'early stopping (arrêt automatique dès que l'AUC de validation cesse de progresser pendant 20 itérations, plutôt qu'un nombre d'itérations choisi à la main). L'entraînement s'arrête à la 87^e itération.

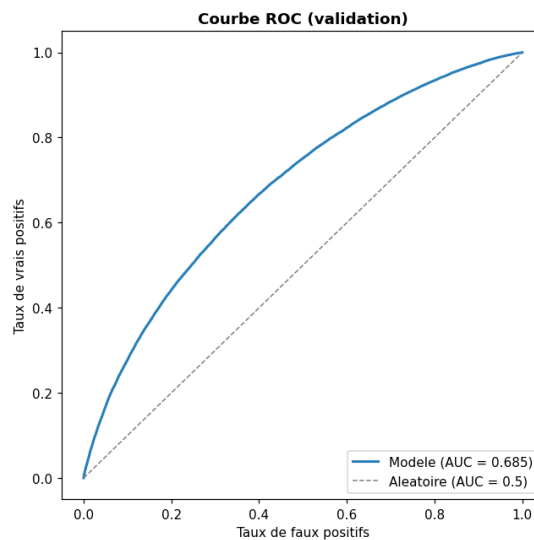
3.5 Recherche d'hyperparamètres (Optuna)

Une recherche d'hyperparamètres a été testée séparément, pour vérifier si elle apportait un gain réel par rapport au modèle par défaut. Une première recherche réduite (14 essais, espace de recherche resserré) donne une AUC de 0.6886 une fois réentraînée à pleine échelle. Une recherche plus large (60 essais) donne 0.6897.

Le gain observé se situe donc entre +0.002 et +0.004 d'AUC selon le nombre d'essais. Cette recherche n'a donc pas été retenue comme modèle principal, mais documentée comme piste testée plutôt qu'ignorée.

4. Résultats

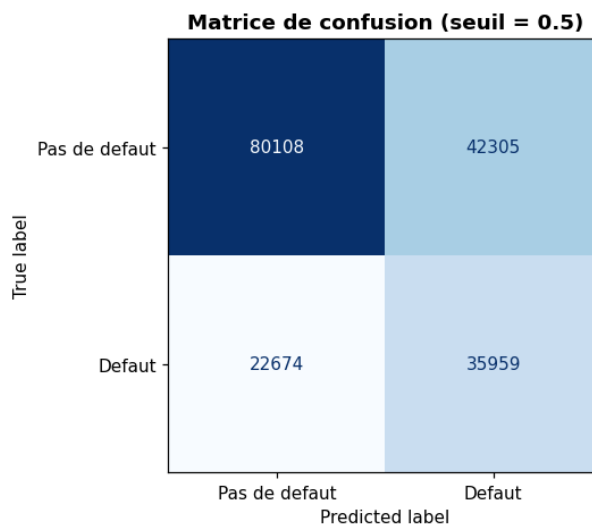
4.1 Performance globale



Courbe ROC - validation, AUC = 0.685

ROC AUC train : 0.7206 - ROC AUC validation : 0.6853.

4.2 Matrice de confusion et rapport de classification

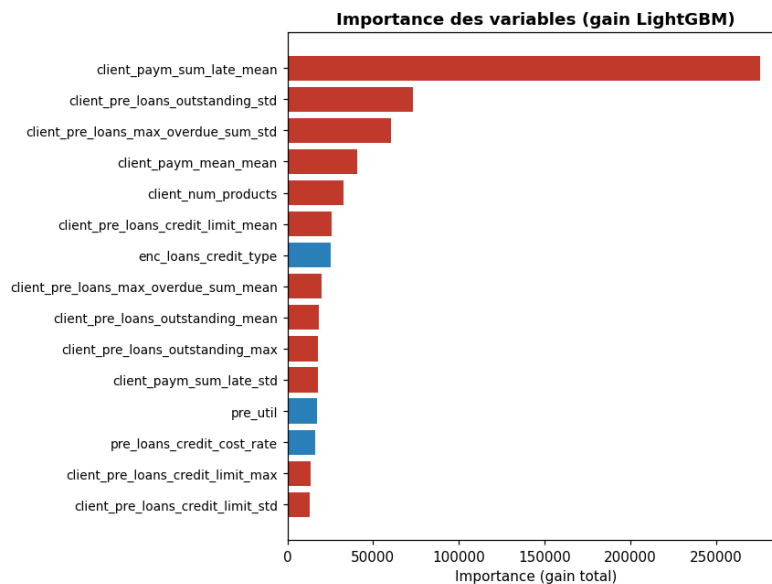


Matrice de confusion - seuil = 0.5, échantillon rééquilibré (~33% de défaut)

Classe	Précision	Rappel	F1-score	Support
Pas de défaut	0.78	0.65	0.71	122 413
Défaut	0.46	0.61	0.53	58 633
Exactitude globale			0.64	181 046

À ce seuil, environ 61% des vrais défauts sont détectés (rappel), pour une précision de 46% sur les alertes. Ce seuil de 0.5 est raisonnable sur l'échantillon rééquilibré utilisé ici, mais une application telle quelle sur la population réelle (3,5% de défaut, non rééquilibrée) nécessiterait une recalibration séparée.

4.3 Importance des variables (gain)

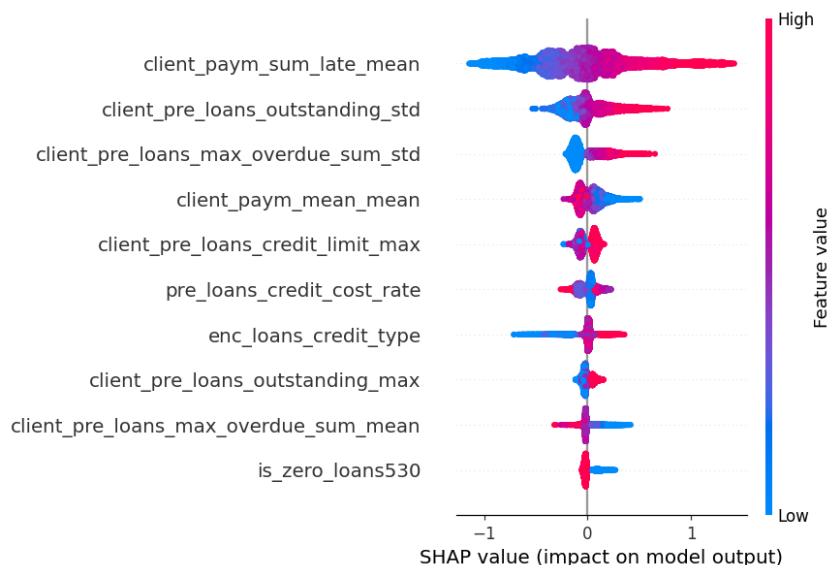


Importance des variables - gain total LightGBM

Les agrégats au niveau client (préfixés client_, en rouge) dominent largement le classement (11 des 15 premières variables) : l'historique complet du client porte plus d'information que les seules caractéristiques du prêt évalué.

4.4 Interprétabilité (SHAP)

L'importance par gain est une moyenne globale : elle ne dit pas dans quel sens une variable pousse la prédiction, ni si son effet est constant. SHAP (bibliothèque shap, TreeExplainer) donne, pour chaque prédiction, une contribution signée par variable.



Distribution des valeurs SHAP - 10 variables les plus influentes

Le classement SHAP confirme celui du gain (agrégats client en tête), un bon signe de cohérence. Le graphique révèle en plus le sens de l'effet : pour `client_paym_sum_late_mean`, les valeurs élevées (rouge) poussent nettement vers le défaut et les valeurs faibles (bleu) à l'inverse. Pour `client_pre_loans_credit_limit_max`, l'effet est plus mélangé, donc moins linéaire. Ce niveau de détail n'est pas visible avec la seule importance par gain.

5. Étape complémentaire : l'historique du client

Un client peut avoir plusieurs prêts dans les données : si l'issue de l'un est connue, c'est un signal fort sur les autres (pratique courante en scoring bureau). Plutôt qu'un post-traitement fragile, cette information est intégrée comme variable du modèle, via un encodage leave-one-out : la moyenne du flag des autres prêts du même client, en excluant la ligne courante (pour éviter la fuite directe qu'impliquerait une moyenne incluant la ligne elle-même).

97,9% des lignes ont au moins un autre prêt connu dans le même sous-ensemble. Le même LightGBM, réentraîné avec ces deux colonnes supplémentaires, fait passer l'AUC de validation de 0.685 à environ 0.9999. Un saut qui doit alerter plutôt que réjouir, et qui est décomposé avant d'être présenté comme un succès :

Sous-ensemble	AUC	Part des lignes
Clients avec historique	1.0 (exact)	97.9%
Clients sans historique	0.68	2.1%

Sur les 97,9% de lignes avec historique, l'AUC de 1.0 n'est pas un exploit de modélisation : flag étant identique pour toutes les lignes d'un client, la variable recopie exactement l'étiquette à partir d'un autre prêt du même client (rien à apprendre). Sur les 2,1% de clients réellement nouveaux, l'AUC de 0.68 est quasi identique au modèle de base : c'est cette valeur, pas 0.9999, qui reflète la vraie capacité du modèle sur un nouveau client.

Cette technique est légitime dans un contexte réel (un établissement connaît l'historique de ses clients existants) pour un client récurrent, mais trompeuse si le score global est présenté sans être décomposé, masquant la vraie capacité du modèle sur un nouveau client, le cas d'usage le plus important en pratique.

6. Analyse critique : deux mécanismes de score artificiellement gonflé

Ce projet met en évidence, et documente délibérément, deux mécanismes distincts par lesquels un score peut être gonflé sur ce type de jeu de données, plutôt que de les exploiter silencieusement pour afficher un chiffre plus impressionnant :

Version	AUC (valid)	Ce qu'elle mesure réellement
Split naïf (contre-exemple, section 3.3)	0.7247	Fuite via des clients partagés entre train et validation => ne reflète pas la vraie capacité du modèle.
Modèle simple, split groupé (section 4.1)	≈ 0.685	Capacité réelle à évaluer un risque à partir des caractéristiques d'un prêt => le cas d'usage le plus utile (nouveau client).
+ historique client LOO (section 5)	≈ 0.9999	Capacité à retrouver le flag via un autre prêt du même client, quand l'info existe déjà => pas un progrès de modélisation.

Dans les deux cas de gonflement (split naïf et encodage LOO), le score élevé mesure la capacité à reconnaître un client déjà vu, pas à évaluer un risque à partir de caractéristiques génériques. Le modèle simple à split groupé (AUC ≈ 0.685) reste donc le résultat principal retenu pour ce projet, car c'est le seul des trois qui généralise à un client réellement nouveau.

7. Limites et pistes d'amélioration

- Entraînement sur le jeu de données complet (17,6M lignes) plutôt que sur l'échantillon de 150 000 clients.
- Validation croisée GroupKFold à K plis plutôt qu'un split unique, pour une estimation plus robuste de l'AUC.
- Ensembling léger (LightGBM + CatBoost + méta-modèle) => gain généralement modeste dans ce type de problème, non testé ici.
- Recalibration des probabilités pour la population réelle (3,5% de défaut) avant tout usage d'un seuil de décision en production.
- En production, exposer séparément les métriques sur les clients déjà connus et sur les nouveaux clients, plutôt qu'un score global unique qui mélangerait les deux cas d'usage.

8. Conclusion

Ce projet retient un modèle LightGBM simple (hyperparamètres par défaut, $AUC \approx 0.685$ en validation) comme résultat principal, obtenu avec une validation groupée et stratifiée par client, sans fuite de données. Deux mécanismes de gonflement artificiel du score. Un split naïf et un encodage d'historique client sont démontrés, mesurés et décomposés plutôt qu'exploités pour afficher un chiffre plus élevé. Une recherche d'hyperparamètres, testée séparément, confirme que le gain disponible au-delà du modèle par défaut reste marginal. L'ensemble du pipeline (notebook complet) est reproductible et documenté.